

# Apuntes de C++

Luciano Arellano

September 3, 2026

## Contents

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Sintaxis basica</b>                                               | <b>2</b>  |
| 1.1      | hello world . . . . .                                                | 2         |
| 1.2      | Compilacion . . . . .                                                | 2         |
| 1.3      | Declaracion, tipos y <i>casting</i> . . . . .                        | 3         |
| 1.4      | Loops, if else, etc. . . . .                                         | 4         |
| <b>2</b> | <b>Funciones</b>                                                     | <b>5</b>  |
| 2.1      | Desafio . . . . .                                                    | 5         |
| 2.1.1    | <i>Skeleton code</i> para el desafio . . . . .                       | 6         |
| <b>3</b> | <b>Manejo de memoria</b>                                             | <b>6</b>  |
| <b>4</b> | <b>Arreglos y Vectores</b>                                           | <b>8</b>  |
| 4.1      | Arreglos . . . . .                                                   | 8         |
| 4.2      | Desafio 2: como esta ordenado un array de dos dimensiones? . . . . . | 9         |
| 4.3      | Strings . . . . .                                                    | 9         |
| 4.4      | Alocacion dinamica de memoria . . . . .                              | 9         |
| 4.5      | Vectores . . . . .                                                   | 9         |
| <b>5</b> | <b>Programacion orientada al objeto (OOP)</b>                        | <b>10</b> |

# 1 Sintaxis basica

## 1.1 hello world

Partimos con un ejemplo. `hello_world.cpp`

```
#include <iostream>
int main() {
    std::cout << "hello world." << std::endl;
    return 0;
}
```

La primera linea importa un *header* (algo análogo a importar un módulo en Python) que en este caso nos permite sacar input/outputs a pantalla.

Nuestro programa en C++ siempre debe tener una (unica) funcion llamada **main**. La funcion debe retornar un numero entero, y es estandar que retorne 0 si el programa se ejecutó correctamente. El *scope* de la funcion esta delimitado por parentesis llave { }.

Esta funcion main tiene dos *statements*, los cuales en C++ siempre deben terminar en ; (punto coma, o *semicolon*). De hecho a diferencia de Python no importan los saltos de linea. El siguiente código también es C++ sintacticamente correcto (por favor no escribir asi!):

```
#include <iostream>
int main() {std::cout << "hello world." << std::endl; return 0;}
```

La linea de output es un poco mas dificil de explicar por ahora, pero podemos imaginar que estamos llamando a una “funcion” `cout` que vive en `std`, como por ejemplo en python podriamos hacer `import numpy as np` y luego `np.sqrt()` para llamar la funcion raiz cuadrado de numpy.

El *namespace* `std` es una coleccion de funciones y objetos. Es comun que la gente escriba

```
#include <iostream>
using namespace std;
int main() {
    cout << "hello world." << endl;
    return 0;
}
```

En este caso nos ahorramos tipear `std::` cada vez que llamamos a alguien de ese namespace (que generalmente es bastantes veces para `std`). El analogo en Python seria `import numpy` a secas. Pero si hacemos esto perdemos de vista donde vive cada sujeto, asi que no lo haremos.

## 1.2 Compilacion

C++ es un lenguaje compilado. Escribimos primero nuestro archivo de texto `hello_world.cpp` y luego compilamos. Aqui “cpp” viene de “C plus plus”, las extensiones usadas son convenciones, pero mejor seguir con `cpp`. Otras usadas son `.cxx` (signos + diagonales), `.cc` (usada en windows?) y `.C` (no usar, es mas para C).

```
g++ hello_world.cpp
```

esto por defecto va a generar un ejecutable llamado `a.out` que podemos ejecutar con

```
./a.out
```

Si queremos darle un nombre usamos el flag de output `-o`

```
g++ hello_world.cpp -o hello
```

que ejecutamos como

```
./hello
```

Si hay un error de sintaxis, muchas veces el compilador nos lo va a decir, pero tambien puede haber *warnings* que, aunque no fatales, nos pueden advertir de algun peligro. A esta altura, nos conviene que el compilador nos muestre todos los warnings con `-Wall`

```
g++ hello_world.cpp -o hello -Wall
```

Si queremos realmente tener un codigo seguro, podemos poner al compilador con el maximo nivel de exigencia con

```
g++ hello_world.cpp -o hello -Wall -pedantic-errors
```

### 1.3 Declaracion, tipos y *casting*

C++ es un lenguaje *estrictamente tipeado* (strongly typed), es decir que cada variable debe tener un tipo bien definido e inmutable. Uno tiene que **declarar** una variable para reservar el espacio de memoria que va a usar, y luego se le **asigna** un valor. En contraste, en *Python* las variables no se declaran.

```
int a; // declaramos "a" como un entero
a = 18; // le asignamos el valor 18
```

Se puede tambien declarar e **inicializar** en un solo statement. Un par de formas comunes son:

```
int a = 18; // muy comun de ver
int b {18}; // forma moderna (desde C++11)
float a {18.0}; // ERROR! "a" ya fue declarado como int
```

En estos casos inicializar es casi lo mismo que asignar, pero no necesariamente. Uno puede tener una rutina de inicializacion mas compleja.

Uno puede definir una variable como constante, lo que impide que su valor sea reasignado. En este caso debe ir inicializada.

```
const float pi {3.1415926}; // la variable pi queda fija
pi = 3.0; // ERROR! no se puede asignar un const
const float b; // Warning o error? (revisen!) const no inicializada
#define PI 3.14 // old school. No usar esta forma.
```

Si asignamos un valor de un tipo a una variable de distinto tipo necesitamos convertirlo, o hacer un *casting*. El compilador puede a veces convieir tipos sin avisarnos: eso un casting implícito.

```
int myint {4};
float myfloat = myint; // ?? cast de int a float?

float myfloat2 {13.8};
int myint2 = myfloat; // ?? float a int? prueben!
```

Para tipos simples el compilador generalmente hace un buen trabajo, pero ustedes deberían querer tener control total sobre esto.

```
double x {18.6};
int n = (int) x; // cast estilo C
int m = static_cast<int>(x); // estilo C++, conversion explicita
```

El error mas tipico es la division enetra. Si uno divide dos int o dos float, C++ usa funciones “division” distintas

```
int ai {1};
int bi {2};
std::cout << ai/bi << std::endl;

float af {1.0};
float bf {2.0};
std::cout << af/bf << std::endl;

// que sucedera con ai/bf? af/bi?
```

A este nivel parece muy sencillo, pero despues vamos a tener cosas mas delicadas

```
Animal *a = ...;
Dog *d = static_cast<Dog*>(a);
```

C++ nos da control absoluto de como hacer castings. Otra ventaja, podemos hacer mucho mas facilmente un Ctrl+f a todas las instancias de `static_cast`.

## 1.4 Loops, if else, etc.

```
// i++ es lo mismo que i+=1
for ( int i {0}; i < 10; i++ ) {
    std::cout << i << std::endl;
}

int b {10};
while ( b > 0; b-- ) {
    std::cout << b << std::endl;
}

int n {10};
if ( 20 > n ) {
    std::cout << "menor que 20" << std::endl;
} else if ( n > 0 ) {
    std::cout << "mayor que 0, menor que 20" << std::endl;
} else {
    std::cout << "menor o igual que 0 o mayor o igual que 20" << std::endl;
}
```

## 2 Funciones

Las funciones en C++ se definen como

```
// esta funcion debe retornar un int
int mi_funcion_int() {
    return 1;
}

// funcion tipo void no retorna nada!
void funcion_void(int arg1) { // toma como argumento un int
    std::cout << "El entero es: " << arg1 << std::endl;
}
```

Uno puede declarar una funcion antes del main, e **implementarla** despues del main():

```
void myfunc(int a);

int main() {
    int a {78};
    myfunc(a);
}

void myfunc(int a) {
    std::cout << "a vale " << a << std::endl;
}
```

Como mencionamos la clase anterior, uno puede **sobrecargar** funciones. Tenemos una funcion que eleva al cuadrado, podriamos usar distintas funciones para elevar int y floats

```
int cuadrado_i(int a) {
    return a*a;
}

float cuadrado_f(float a) {
    return a*a;
}
```

pero tiene mucho mas sentido que uno haga una pura funcion, que es elevar al cuadrado.

```
int cuadrado(int a) {
    return a*a;
}

float cuadrado(float a) {
    return a*a;
}
```

### 2.1 Desafio

Sobrecargue la funcion `cuadrado` para que calcule el cuadrado de una matriz  $2 \times 2$ . La funcion debe retornar tipo `void` y sacar por pantalla los digitos en orden correcto.

Ejecute la funcion para las matrices

- $m_1 = \begin{pmatrix} 2 & 3 \\ 4 & 5 \end{pmatrix}$
- $m_2 = \begin{pmatrix} -1 & 0 \\ -2 & 1 \end{pmatrix}$
- $m_3 = \begin{pmatrix} 2.1 & 0.7 \\ -7.2 & 0.6 \end{pmatrix}$

Ejemplo de output para  $m_I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

```
Cuadrado de m_I:
1 0
0 1
```

### 2.1.1 *Skeleton code* para el desafío

```
#include <iostream>

// int
int cuadrado(int a) {
    std::cout << "Llamaste a la funcion int\n";
    return a*a;
}

// float
float cuadrado(float a) {
    std::cout << "Llamaste a la funcion float\n";
    return a*a;
}

// matrix
/*
Implementacion de la funcion:
deben definir cuantos argumentos va a tener
*/
void cuadrado(){
    std::cout << "Llamaste a la funcion matrix\n";
}

int main() {
    int a {9};
    std::cout << cuadrado(a) << std::endl;
    float b {3.2};
    std::cout << cuadrado(b) << std::endl;
    // cuadrado(matrix??);
    return 0;
}
```

## 3 Manejo de memoria

En la clase anterior hemos aprendido el rol de la memoria en un computador. Ahora vemos que en C++ tenemos que declarar cada variable, dejando fijo su tipo. Al inicializar

una variable, la direccion de memoria que ocupa queda fija. Podemos acceder a ella con el operador &

```
int a {10};
std::cout << "El valor de a es " << a << std::endl;
std::cout << "La direccion de memoria de a es " << &a << std::endl;
```

El valor de a es 10

La direccion de memoria de a es 0x7fffd31e8844

Noten que con cada ejecucion la direccion de memoria cambia. Realmente nos estamos metiendo en el cerebro del computador.

Podemos declarar una variable como **referencia** a otra,

```
int a {10};
int &r {a}; // r es una referencia a "a"
r = 30;     // r ES a
std::cout << "El valor de a es " << a << std::endl;
```

Esto es particularmente util para funciones, si queremos que una funcion cambie nuestros numeritos, le pasamos valores por referencia

```
int by_value( int a ) { // hace una copia de la variable que le pasamos
    a = a+10; // le suma 10 a la copia
    return a; // devuelve la copia
}

void by_reference (int &a ) { // le pasamos la variable de verdad
    a = a+10; // nos cambia la variable original
}

int main () {
    int a {10};
    int b;
    b = by_value(a);
    std::cout << "a, b: " << a << " " << b << std::endl;
    by_reference(a);
    std::cout << "a: " << a << std::endl;
}
```

Esto es tremendamente poderoso! una vez que sabemos la direccion de memoria, podemos trabajar directamente con ella a traves de un **puntero** (*pointer*).

```
int *b; // b es un puntero de enteros
std::cout << "El valor del puntero b es " << b << std::endl;
b = &a;
std::cout << "El valor del puntero b es " << b << std::endl;
std::cout << "b apunta al valor " << *b << std::endl;
```

El valor del puntero b es 0x7f512834548f

El valor del puntero b es 0x7fffd31e8844

b apunta al valor 10

Notemos que inicialmente b apunta a cualquier lado. Pero al asignar `b = &a` ahora apunta a la direccion de memoria de a. Y entonces podemos *dereferenciar* b usando `*b` para obtener el valor int guardado en memoria. No solo podemos obtener el valor en

memoria, podemos cambiarlo. `a` no es dueño de la memoria, nosotros tenemos control total

```
int a {10};
std::cout << "El valor de a es " << a << std::endl;
int *b {&a}; // b es un puntero hacia a
*b = 20;      // el valor hacia el cual b apuntaba ahora es 20
std::cout << "El valor de a es " << a << std::endl;
```

Cuidemos el lenguaje con la siguiente tabla:

| Usado en \ operador | <code>&amp;</code>   | <code>*</code> |
|---------------------|----------------------|----------------|
| Declaracion         | Referencia           | Puntero        |
| Expresion           | Direccion de memoria | Dereferencia   |

## 4 Arreglos y Vectores

### 4.1 Arreglos

Como vimos la clase anterior, un arreglo es una estructura de datos basica tremendamente eficiente (lectura  $\mathcal{O}(1)$ ) pero que tiene un tamaño fijo. Se reservan  $N$  bloques de memoria contiguos en el momento de declaracion,

```
int a[3];
a[0] = 1;
a[1] = 2;
a[2] = -1;

float b[2] = { -2.3, 4.5 };

for ( int i = 0; i < 3; i++ ) {
    std::cout << "a[" << i << "]: " << a[i] << std::endl;
    std::cout << "b[" << i << "]: " << b[i] << std::endl;
    // notemos que b sale del rango
}
```

En este ejemplo definimos `b` con largo 2 pero le pedimos el tercer elemento. El compilador no se preocupa de esto ya que no le estamos pidiendo algo invalido. Esta falta de checks es un arma de doble filo. Es lo que permite que los arreglos en C++ sean extremadamente eficientes, tener pocos chequeos.

Un array es realmente un puntero. Si un float son 4 bytes, el arreglo lo unico que fija es la primera direccion de memoria, y luego va sumando 4 cada vez. Supongamos que la primera direccion es `&b[0] → 1320`, entonces `&b[1] → 1324`, y en general `&b[n] → 1320 + 4 * n`. Veamoslo explicitamente

```
const int nentries {5};
float array[nentries];
for ( int i {0}; i < nentries; i++ ) {
    array[i] = 1.1*i;
    std::cout << i << " " << array[i] << " " << *(array+i) << std::endl;
}
```

vemos que `*array` apunta a la primera direccion de memoria, luego la  $i$ -esima direccion se obtiene con `*(array+i)`.



## 4.2 Desafio 2: como esta ordenado un array de dos dimensiones?

```
int a[2][3]; // ??
```

## 4.3 Strings

Un `string` es una cadena de texto (en general). En C++ se implementa como un arreglo de caracteres. Vimos que los caracteres son la unidad basica, cada uno ocupa un byte en memoria. No vamos a ahondar mucho con strings en C++ porque despues en ROOT vamos a tener mejores maneras de trabajar con strings.

```
#include <iostream>
#include <string>

int main() {

    char a {'b'};
    char c {'c'};
    std::string mystring{"hello world"};

    for ( int i {0}; i < mystring.length(); i++ ) {
        std::cout << i << " " << mystring[i] << std::endl;
    }
    return 0;
}
```

## 4.4 Alocacion dinamica de memoria

Uno debe definir el tamano de los arreglos al momento de declarar, pero no siempre lo sabemos. Primero, veamos como generar numeros aleatorios.

```
#include <iostream>
#include <random>

int main() {
    std::mt19937 gen {2399}; // semilla 2399
    std::uniform_int_distribution<int> dist {0, 10};
    for ( int i {0}; i < 15; i++ ) {
        int nparticles { dist(gen) };
        std::cout << nparticles << std::endl;
    }
    return 0;
}
```

Aqui usamos `mt19937` (generador Mersenne Twister) para generar numeros pseudo-aleatorios a partir de una semilla. Tenemos una distribucion uniforme de enteros entre 0 y 10. La funcion `dist(gen)` nos entrega los numeros aleatorios. Supongamos ahora que en cada evento tenemos una cantidad `dist(gen)` de particulas (`int nparticles { dist(gen) }`) y queremos hacer un arreglo con las energias de todas las particulas en cada evento. Si tratamos de hacer un arreglo poniendo `nparticles`, el compilador nos deja, pero

## 4.5 Vectores

Los vectores son tremendamente utiles.

```

#include <iostream>
#include <vector>

int main() {
    std::vector<int> v1;                // vector de int (vacío)
    std::vector<float> v2 { 3.3, 1/34.}; // vector de floats (3 inicialmente)
    v1.push_back(4);                   // le agregamos un elemento a v1 (al final)

    for ( int i {0}; i < v1.size(); i++ ) {
        std::cout << "v1[" << i << "]: " << v1[i] << std::endl;
    }

    for ( int i {0}; i < v2.size(); i++ ) {
        std::cout << "v2[" << i << "]: " << v2[i] << std::endl;
    }
    return 0;
}

```

## 5 Programación orientada al objeto (OOP)

Se viene.